

SISTEMAS DISTRIBUIDOS

API REST de Inscrições em Oficinas

Atividade Prática II — Comunicação baseada em recursos

API REST para gestão de oficinas com vagas limitadas e das inscrições de participantes nelas. O domínio foi escolhido por possuir conflitos de concorrência intrínsecos — duas pessoas podem disputar a última vaga —, o que torna observáveis os problemas de idempotência, perda de atualização e distinção entre falha do servidor e ausência de resposta.

ALUNO

Antonio Carlos Silva Junior

REPOSITÓRIO

github.com/antonio-carlosjr/SistemasDistribuidos

DATA

31/08/2026

Sumário

1. Visão geral e entregáveis

2. Modelagem de recursos

3. Execução e endpoints

4. Tabela de evidências

5. Demonstração prática

6. Análise

Documento gerado por `gerar_entrega.py` a partir dos arquivos versionados no repositório. As capturas de tela e a tabela de evidências resultam de execuções reais, não de transcrição manual.

1. Visão geral e entregáveis

O domínio

A API gerencia **oficinas** com vagas limitadas e as **inscrições** de participantes nelas. O domínio é distinto do exemplo da apostila (dispositivos/leituras), conforme exigido, e foi escolhido por três motivos:

1. Possui **duas coleções com dependência real** — uma inscrição não existe sem a oficina a que pertence.
2. Produz **conflitos de naturezas diferentes**: capacidade esgotada e violação de unicidade são ambos 409, mas por razões distintas.
3. Tem **disputa natural por um recurso escasso** — a última vaga —, o que torna demonstráveis os problemas de concorrência que a disciplina discute.

Entregáveis

#	Entregável exigido	Onde está nesta entrega
1	Código-fonte organizado	Repositório, seção 3 descreve a estrutura
2	README com instruções de execução	Seção 3
3	Coleção de testes/curl ou script cliente	Seção 3 e 5; suíte pytest, <code>cliente_testes.py</code> e os scripts em <code>demo/</code>
4	Tabela cenário / requisição / status esperado / obtido	Seção 4
5	Demonstração prática curta	Seção 5, com capturas de cada passo

As questões de análise, as justificativas de URI, método e status, e a modelagem prévia dos recursos estão nas seções 6 e 2.

Requisitos obrigatórios

Requisito	Atendimento
Pelo menos duas coleções relacionadas	<code>/oficinas</code> e <code>/oficinas/{id}/inscricoes</code>
No mínimo seis endpoints, com GET, POST, PUT ou PATCH e DELETE	14 endpoints: 11 de domínio e 3 operacionais
Validação de entrada	Pydantic com restrições por campo e <code>extra="forbid"</code>
Tratamento explícito de 404	Oficina e inscrição inexistentes, em ambos os níveis
Tratamento explícito de 409	Seis regras distintas de conflito
Cliente de teste programático com timeout definido	<code>cliente_testes.py</code> , com <code>timeout</code> em toda chamada
Persistência em arquivo ou banco	Arquivo JSON com escrita atômica e exclusão mútua

Roteiro mínimo

Passo	Cumprimento
1. Modelar recursos e relações antes de programar	Seção 2, escrita e versionada antes do código
2. Implementar a API e gerar dados de teste	<code>seed.py</code> prepara os cinco estados da demonstração
3. Criar roteiro automatizado de testes	40 testes pytest e 48 cenários no cliente programático
4. Provocar dois erros de aplicação e uma falha de conectividade	Seis conflitos, além de 404, 422, 412, 428, 503 e <code>ConnectionError</code>
5. Coletar logs e organizar evidências	Log estruturado por requisição, tabela gerada e capturas de tela

Desafios opcionais

Desafio	Situação
ETag / versionamento otimista	Implementado, com 428, 412 e 304
Autenticação baseada em token	Parcial — protege apenas o endpoint administrativo
Dois clientes concorrentes observando conflitos	Implementado em <code>cliente_concorrente.py</code> , com dois experimentos

Resultados verificados

Verificação	Resultado
Suíte automatizada	40 de 40 testes aprovados
Cenários do cliente programático	48 de 48 com status obtido igual ao esperado
Disputa de 12 clientes por 1 vaga	Exatamente um 201 e onze 409; capacidade preservada
Perda de atualização	Detectada com 412; a primeira escrita sobrevive

Os dois testes de concorrência foram validados por contraprova: substituindo o lock do repositório por um contexto vazio, ambos passam a falhar. Eles não passam por acaso.

Limites conhecidos

Três limitações são registradas explicitamente, porque uma análise que só descreve acertos é incompleta:

1. **A persistência não é transacional.** As escritas são atômicas por arquivo, e não por operação lógica.
2. **A coordenação não atravessa processos.** O lock protege threads de uma única instância; duas instâncias sobre o mesmo arquivo não estariam protegidas.
3. **Não há autenticação de participantes.** Qualquer cliente pode cancelar qualquer inscrição; o token cobre apenas o endpoint administrativo.

Nenhuma das três é resolvida por usar REST ou HTTP — o que é, em si, a observação central da atividade.

2. Modelagem de recursos

Documento escrito **antes** da implementação, conforme o passo 1 do roteiro mínimo ("Modele recursos e relações antes de programar").

Domínio

Gestão de **oficinas** com vagas limitadas e as **inscrições** de participantes nelas. O domínio é distinto do exemplo da apostila (dispositivos/leituras) e foi escolhido por possuir conflitos de concorrência intrínsecos: duas pessoas podem disputar a última vaga.

Recursos

Oficina

Uma sessão com título, instrutor, horário e um número finito de vagas.

Campo	Tipo	Regras
id	inteiro	Atribuído pelo servidor
titulo	texto	3 a 120 caracteres
instrutor	texto	3 a 80 caracteres
vagas_totais	inteiro	1 a 500
inicio	ISO-8601 UTC	Instante de início
duracao_min	inteiro	15 a 480 minutos
status	enumeração	rascunho , aberta , encerrada , cancelada
versao	inteiro	Incrementado a cada escrita; base do ETag
vagas_ocupadas	inteiro	Derivado — contagem de inscrições ativas

`vagas_ocupadas` não é armazenado. Se fosse, existiriam duas fontes de verdade para a mesma informação (o contador e a lista de inscrições), que poderiam divergir sob concorrência ou falha parcial de escrita. Calcular sob demanda elimina a classe inteira de bugs de contador dessincronizado.

`versao` existe para o controle otimista de concorrência e é exposto como `ETag`.

Ciclo de vida do status



Só oficinas em `aberta` aceitam novas inscrições. As demais transições produzem conflito.

Inscrição

A participação de uma pessoa em uma oficina.

Campo	Tipo	Regras
<code>id</code>	inteiro	Atribuído pelo servidor
<code>oficina_id</code>	inteiro	Referência à oficina
<code>participante_nome</code>	texto	3 a 120 caracteres
<code>participante_email</code>	texto	Validado por padrão de expressão regular
<code>status</code>	enumeração	<code>confirmada</code> , <code>cancelada</code> , <code>presente</code>
<code>criada_em</code>	ISO-8601 UTC	Instante do registro

Inscrições `confirmada` e `presente` ocupam vaga; `cancelada` libera a vaga.

`participante_email` usa `Field(pattern=...)` em vez de `EmailStr` do Pydantic, que exigiria a dependência externa `email-validator` sem agregar nada à atividade.

Ciclo de vida do status



`cancelada` é terminal: uma inscrição cancelada não volta a ocupar vaga. Reingressar exige criar uma nova inscrição, o que passa novamente pela verificação de capacidade — caso contrário, um cancelamento seguido de reativação poderia furar o limite de vagas.

Relação entre os recursos

Uma oficina possui muitas inscrições. Uma inscrição **não existe sem** a oficina: é um recurso subordinado, e não uma entidade independente que apenas referencia outra.

Isso se reflete no desenho das URIs:

Operação	URI	Motivo
Criar e listar inscrições	<code>/oficinas/{id}/inscricoes</code>	A coleção só faz sentido no contexto do pai; o identificador do pai é necessário para verificar capacidade
Obter, alterar e cancelar uma inscrição	<code>/inscricoes/{id}</code>	O identificador da inscrição já a localiza sozinho

A escolha de expor as operações de item em primeiro nível é deliberada. Se o cancelamento exigisse `/oficinas/{oficina_id}/inscricoes/{id}`, todo cliente que possuísse apenas o identificador da inscrição — o que o `Location` da criação lhe devolve — precisaria também guardar o identificador do pai. Isso é acoplamento desnecessário: a URI passaria a codificar uma informação que o servidor já conhece.

O custo dessa escolha é que existem dois caminhos relacionados ao mesmo conceito. O `Location` devolvido na criação resolve a ambiguidade ao indicar qual é a URI canônica do item.

Invariantes do domínio

Estas são as propriedades que a API precisa preservar sob qualquer sequência de requisições, inclusive concorrentes:

1. O número de inscrições ativas de uma oficina nunca excede `vagas_totais`.
2. Um mesmo e-mail não possui duas inscrições ativas na mesma oficina.
3. Inscrições só são criadas em oficinas com status `aberta`.
4. Nenhuma oficina com inscrições ativas é removida.
5. `vagas_totais` nunca é reduzido abaixo do número de inscrições ativas.
6. Requisição rejeitada não deixa efeito colateral no estado.

O invariante 1 é o mais interessante: verificá-lo e depois inserir são duas operações distintas, e sem exclusão mútua dois clientes podem passar pela verificação antes que qualquer um dos dois grave. É o padrão *check-then-act*, e é exatamente o que o experimento de concorrência tenta provocar.

Mapeamento para semântica HTTP

Regra violada	Status	Justificativa
Corpo sintaticamente válido, semanticamente inválido	422	Representação bem formada que não satisfaz as restrições do modelo
Recurso inexistente	404	Nada há na URI solicitada
Invariantes 1 a 5	409	A requisição é válida em si, mas conflita com o estado atual do recurso
<code>If-Match</code> ausente em escrita de oficina	428	O servidor exige a précondição para evitar perda de atualização
<code>If-Match</code> divergente	412	A précondição foi enviada e falhou: houve escrita concorrente

A distinção entre 422 e 409 é a que mais se erra: 422 é sobre a **representação enviada**, 409 é sobre o **estado do servidor**. Uma mesma requisição pode ser aceita agora e conflitar um segundo depois sem que nada nela tenha mudado.

3. Execução e endpoints

Atividade Prática II de Sistemas Distribuídos. API REST para gestão de oficinas com vagas limitadas e das inscrições de participantes nelas.

O domínio é diferente do exemplo da apostila (dispositivos/leituras) e foi escolhido por ter conflitos de concorrência intrínsecos: duas pessoas podem disputar a última vaga, e é isso que torna observáveis os problemas que a disciplina discute.

Execução

Requer Python 3.10 ou superior.

```
python -m venv .venv
```

Ative o ambiente virtual — no Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

No Linux ou macOS:

```
source .venv/bin/activate
```

Instale as dependências:

```
pip install -r requirements.txt
```

Gere os dados de demonstração **antes** de subir o servidor — a API carrega o estado do arquivo na inicialização, então semear com o servidor no ar não teria efeito sobre o processo já em execução:

```
python seed.py
```

Suba a API:

```
fastapi dev app/main.py
```

Se o comando acima falhar com `UnicodeEncodeError`, o problema é o banner colorido da CLI do FastAPI, que não consegue ser escrito no console do Windows em cp1252 — a aplicação em si não é afetada. Use a alternativa clássica, que também é a citada pela apostila:

```
python -m uvicorn app.main:app --reload
```

A documentação interativa fica em <http://127.0.0.1:8000/docs>. Ela é útil para exploração, mas os testes abaixo usam linha de comando e cliente programático para deixar a mensagem HTTP explícita.

Endpoints

Oficinas

Método	Caminho	Descrição	Status
GET	/oficinas	Lista paginada, filtrável por <code>status</code>	200, 422
POST	/oficinas	Cria oficina	201, 422
GET	/oficinas/{id}	Obtém oficina	200, 304, 404
PUT	/oficinas/{id}	Substitui a representação	200, 404, 409, 412, 422, 428
PATCH	/oficinas/{id}	Altera parcialmente	200, 404, 409, 412, 422, 428
DELETE	/oficinas/{id}	Remove oficina	204, 404, 409, 412, 428

Inscrições

Método	Caminho	Descrição	Status
GET	/oficinas/{id}/inscricoes	Lista as inscrições da oficina	200, 404
POST	/oficinas/{id}/inscricoes	Inscreve participante	201, 404, 409, 422
GET	/inscricoes/{id}	Obtém inscrição	200, 404
PATCH	/inscricoes/{id}	Altera o status	200, 404, 409, 422
DELETE	/inscricoes/{id}	Cancela e libera a vaga	204, 404

Operacionais

Existem para tornar reproduzíveis os experimentos da seção 4.8 da apostila.

Método	Caminho	Descrição
GET	/saude	200 normal, 503 em manutenção
POST	/admin/manutencao	Liga/desliga manutenção (exige <code>X-Token-Admin</code>)
GET	/debug/lento?segundos=2	Atraso deliberado, para exercer timeouts

Conflitos tratados (409)

Seis regras distintas produzem conflito. Todas dependem do **estado atual** do recurso, e não da representação enviada — é o que as separa do 422:

Código	Situação
<code>oficina_lotada</code>	Não há vagas disponíveis
<code>inscricao_duplicada</code>	O e-mail já tem inscrição ativa nesta oficina
<code>oficina_nao_aberta</code>	A oficina não está com status <code>aberta</code>
<code>oficina_com_inscricoes</code>	Remoção de oficina com participantes ativos
<code>vagas_abaixo_do_ocupado</code>	Redução de <code>vagas_totais</code> abaixo do ocupado
<code>transicao_invalida</code>	Mudança de status fora do ciclo de vida

Controle de concorrência

Escritas em oficina exigem o cabeçalho `If-Match` com o ETag obtido na leitura. Sem a precondição, dois clientes que leram a mesma versão sobrescreveriam um ao outro sem que nenhum percebesse.

Situação	Status
<code>If-Match</code> ausente	428 Precondition Required
<code>If-Match</code> divergente	412 Precondition Failed
<code>If-None-Match</code> igual à versão atual	304 Not Modified

O ETag é derivado de um contador de versão incrementado a cada escrita.

Testando com curl

```
curl -i http://127.0.0.1:8000/oficinas
```

Criar uma oficina — a resposta traz `Location` e `ETag`:

```
curl -i -X POST http://127.0.0.1:8000/oficinas -H "Content-Type: application/json" -d "{\n  \"titulo\": \"Filas e mensageria\",\n  \"instrutor\": \"Ana Reis\",\n  \"vagas_totais\": 2,\n  \"inicio\": \"2026-09-10T14:00:00Z\",\n  \"duracao_min\": 120,\n  \"status\": \"aberta\"}"
```

Inscriver um participante:

```
curl -i -X POST http://127.0.0.1:8000/oficinas/1/inscricoes -H "Content-Type: application/json" -d '{"participante_nome":"Bruno Lima","participante_email":"bruno@exemplo.com"}'
```

Repetir o comando acima devolve **409**: o mesmo e-mail já tem inscrição ativa.

Alterar sem precondition devolve **428**:

```
curl -i -X PATCH http://127.0.0.1:8000/oficinas/1 -H "Content-Type: application/json" -d '{"titulo":"Titulo novo"}'
```

Alterar com o ETag correto devolve **200**:

```
curl -i -X PATCH http://127.0.0.1:8000/oficinas/1 -H "Content-Type: application/json" -H "If-Match: \"1\"" -d '{"titulo":"Titulo novo"}'
```

Testes

Suíte automatizada

Roda em processo, sem servidor. Cobre contrato, códigos de status e os invariantes da seção 4.13.

```
python -m pytest -v
```

Tabela de evidências

Com o servidor no ar, executa os cenários por HTTP real e **gera** `evidencias/tabela.md`:

```
python cliente_testes.py
```

Em seguida, desligue o servidor e execute a variante offline. Ela acrescenta os cenários de falha de conectividade à mesma tabela, mantendo a numeração contínua:

```
python cliente_testes.py --offline
```

A tabela é gerada a partir da execução, e não redigida à mão: uma tabela escrita manualmente descreve o que se acredita que a API faz, esta descreve o que ela fez.

Experimentos de concorrência

```
python cliente_concorrente.py
```

Doze clientes disputam uma única vaga e dois clientes editam a mesma oficina a partir da mesma leitura. A saída registrada está em `evidencias/concorrencia.txt`.

Demonstração passo a passo

`docs/demonstracao.md` traz o roteiro cronometrado da apresentação, com a captura de tela de cada passo. Os cenários de HTTP ficam em `demo/`, um script por conceito — cada um imprime o comando antes de executá-lo:

```
.\demo\04-precondicao.ps1
```

Estrutura

app/	
config.py	configuração por variável de ambiente
modelos.py	modelos Pydantic, enums e ciclos de vida
erros.py	exceções de domínio e mapeamento para status HTTP
repositorio.py	persistência JSON com escrita atômica e exclusão mútua
observabilidade.py	log estruturado com identificador de correlação
main.py	rotas, middlewares e tratadores de erro
tests/	suíte pytest: contrato e invariantes
docs/	
modelagem.md	recursos e relações, escrito antes do código
analise.md	respostas às questões da atividade
demonstracao.md	roteiro da apresentação, com capturas de cada passo
demo/	scripts curl da demonstração, um por cenário
evidencias/	
tabela.md	cenário, requisição, status esperado e obtido
concorrencia.txt	saída dos experimentos concorrentes
servidor.log	uma linha JSON por requisição
capturas/	capturas de tela de cada passo da demonstração
seed.py	dados de demonstração
cliente_testes.py	cliente programático que gera a tabela
cliente_concorrente.py	experimentos de concorrência

O estado de execução fica em `dados/estado.json`, fora do controle de versão.

Observabilidade

Cada requisição gera uma linha JSON em `evidencias/servidor.log` e no stdout:

```
{"ts": "...", "request_id": "3f31109f4407", "metodo": "GET", "caminho": "/oficinas", "status": 200, "duracao_ms": 1.24}
```

O `request_id` vem do cabeçalho `X-Request-Id` quando o cliente o envia, e é gerado quando não vem. Ele volta na resposta e aparece no corpo de erro, ligando o que o cliente viu à linha correspondente no servidor. Credenciais nunca são registradas.

Configuração

Variável	Padrão	Uso
OFICINAS_DB	dados/estado.json	Arquivo de estado
OFICINAS_LOG	evidencias/servidor.log	Log estruturado
OFICINAS_TOKEN_ADMIN	token-de-laboratorio	Token do endpoint administrativo

O token tem valor padrão por ser um laboratório. Em produção um segredo nunca teria default embutido no código.

4. Tabela de evidências

Gerada por `cliente_testes.py` a partir da execucao real contra o servidor. Os valores da coluna *obtido* nao sao transcritos a mao.

48 de 48 cenarios com resultado igual ao esperado.

Sucesso

#	Cenario	Requisicao	Esperado	Obtido	Codigo	Resultado
1	Sonda de saude	GET /saude	200	200	-	OK
2	Criar oficina	POST /oficinas	201	201	-	OK
Responde Location e ETag						
3	Criar a mesma oficina de novo	POST /oficinas	201	201	-	OK
POST nao e idempotente: cria um segundo recurso						
4	Listar oficinas	GET /oficinas	200	200	-	OK
5	Obter oficina	GET /oficinas/6	200	200	-	OK
6	Listar inscricoes da oficina	GET /oficinas/6/inscricoes	200	200	-	OK

Validacao de entrada (422)

#	Cenario	Requisicao	Esperado	Obtido	Codigo	Resultado
7	Criar oficina com titulo curto	POST /oficinas	422	422	entrada_invalida	OK
8	Criar oficina com campo desconhecido	POST /oficinas	422	422	entrada_invalida	OK
<i>Campo nao previsto e erro, nao silencio</i>						
9	Inscriver com e-mail malformato	POST /oficinas/6/inscricoes	422	422	entrada_invalida	OK
10	Listar com limite acima do teto	GET /oficinas?limite=1000	422	422	entrada_invalida	OK
<i>Paginacao tem teto para a resposta nao crescer sem limite</i>						

Recurso inexistente (404)

#	Cenario	Requisicao	Esperado	Obtido	Codigo	Resultado
11	Obter oficina inexistente	GET /oficinas/9999	404	404	nao_encontrado	OK
12	Listar inscricoes de oficina inexistente	GET /oficinas/9999/inscricoes	404	404	nao_encontrado	OK
<i>404, e nao lista vazia: sao situacoes distintas</i>						
13	Obter inscricao inexistente	GET /inscricoes/9999	404	404	nao_encontrado	OK

Concorrencia e cache (304/412/428)

#	Cenario	Requisicao	Esperado	Obtido	Codigo	Resultado
14	Obter com If-None-Match atual	GET /oficinas/6	304	304	-	OK
<i>Representacao inalterada: resposta sem corpo</i>						
15	Alterar sem If-Match	PATCH /oficinas/6	428	428	precondicao_obrigatoria	OK
<i>A precondicao e exigida para evitar perda de atualizacao</i>						
16	Alterar com ETag inexistente	PATCH /oficinas/6	412	412	precondicao_falhou	OK
17	Alterar com ETag correto	PATCH /oficinas/6	200	200	-	OK
<i>A versao avanca e o ETag anterior deixa de valer</i>						
18	Repetir a alteracao com o ETag ja consumido	PATCH /oficinas/6	412	412	precondicao_falhou	OK
<i>Perda de atualizacao detectada em vez de sobrescrita silenciosa</i>						
19	Reler para obter o ETag vigente	GET /oficinas/6	200	200	-	OK
20	Substituir com If-Match vigente	PUT /oficinas/6	200	200	-	OK

Conflitos de dominio (409)

#	Cenário	Requisicao	Esperado	Obtido	Codigo	Resultado
21	Inscrever participante	POST /oficinas/6/inscricoes	201	201	-	OK
22	Inscrever o mesmo e-mail de novo	POST /oficinas/6/inscricoes	409	409	inscricao_duplicada	OK
	<i>Repetir POST aqui conflita, em vez de duplicar</i>					
23	Inscrever um segundo participante	POST /oficinas/6/inscricoes	201	201	-	OK
	<i>Leva a oficina a duas vagas ocupadas de tres</i>					
24	Remover oficina com inscritos	DELETE /oficinas/6	409	409	oficina_com_inscricoes	OK
	<i>409 e nao 403: o impedimento vem do estado, nao da permissao</i>					
25	Reduzir vagas abaixo do ocupado	PATCH /oficinas/6	409	409	vagas_abaixo_do_ocupado	OK
	<i>Ha duas inscricoes ativas; aceitar deixaria estado impossivel</i>					
26	Transicao de status invalida	PATCH /oficinas/6	409	409	transicao_invalida	OK
27	Criar oficina de vaga unica	POST /oficinas	201	201	-	OK
28	Ocupar a unica vaga	POST /oficinas/8/inscricoes	201	201	-	OK

Timeout do cliente

#	Cenario	Requisicao	Esperado	Obtido	Codigo	Resultado
40	Rota de 2s com timeout de 0.5s	GET /debug/lento?segundos=2 (timeout 0.5s)	Timeout	Timeout	-	OK
41	Rota de 2s com timeout de 1.0s	GET /debug/lento?segundos=2 (timeout 1.0s)	Timeout	Timeout	-	OK
42	Rota de 2s com timeout de 3.0s	GET /debug/lento?segundos=2	200	200	-	OK

Indisponibilidade (503)

#	Cenario	Requisicao	Esperado	Obtido	Codigo	Resultado
43	Ativar manutencao sem token	POST /admin/manutencao	401	401	nao_autorizado	OK
44	Ativar manutencao com token	POST /admin/manutencao	200	200	-	OK
45	Listar oficinas em manutencao	GET /oficinas	503	503	em_manutencao	OK
Ha resposta: status e Retry-After informam o cliente						
46	Desativar manutencao	POST /admin/manutencao	200	200	-	OK

Falha de conectividade

#	Cenario	Requisicao	Esperado	Obtido	Codigo	Resultado
47	Listar oficinas com o servidor desligado	GET /oficinas	ConnectionError	ConnectionError	-	OK
Nao ha resposta: nem status, nem corpo, nem Retry-After						
48	Criar oficina com o servidor desligado	POST /oficinas	ConnectionError	ConnectionError	-	OK
O cliente nao sabe se a operacao ocorreu						

5. Demonstração prática

Sequência para a demonstração prática, com as capturas de cada passo já executado. Duração alvo: **7 minutos**.

As imagens são capturas reais dos comandos rodando nesta máquina, e servem para dois propósitos: ensaiar a apresentação sabendo exatamente o que vai aparecer na tela, e servir de prova caso algo falhe no dia.

Antes de começar

Dois terminais abertos na pasta `Atividade_Prática_II`, com o ambiente virtual ativado nos dois:

Terminal	Papel
A	Servidor. Fica rodando e mostrando o log estruturado
B	Cliente. É onde os comandos da demonstração são digitados

Confira antes de apresentar:

- ☐ `pip install -r requirements.txt` já executado
- ☐ Porta 8000 livre
- ☐ `python seed.py` roda sem erro
- ☐ Os cinco scripts em `demo/` executam

Ordem importa. `seed.py` precisa rodar **antes** de subir o servidor. A API carrega o estado do arquivo na inicialização, então semear com o servidor no ar não teria efeito sobre o processo em execução.

Passo 1 — Estado inicial (30 s)

Terminal B:

```
python seed.py
```

```
AP2-demo

PS .venv\Scripts\python.exe seed.py

oficina 1 aberta      20 vagas Introducao a Kubernetes
oficina 2 aberta      1 vagas Observabilidade com OpenTelemetry
oficina 3 aberta      2 vagas Filas e mensageria com RabbitMQ
oficina 4 rascunho    30 vagas Consenso distribuido e Raft
oficina 5 encerrada   15 vagas Padroes de resiliencia em microservicos
inscricao 1 na oficina 3 fernanda.dias@exemplo.com
inscricao 2 na oficina 3 gabriel.matos@exemplo.com

Estado pronto para a demonstracao:
oficina 1 -- aberta e vazia (cenarios de sucesso)
oficina 2 -- aberta com 1 vaga (disputa concorrente)
oficina 3 -- lotada (409 oficina_lotada)
oficina 4 -- rascunho (409 oficina_nao_aberta)
oficina 5 -- encerrada (409 oficina_nao_aberta)

■
```

O que dizer: as cinco oficinas cobrem os estados que a demonstração precisa. A de vaga única é onde os clientes concorrentes vão disputar; a lotada e a em rascunho produzem conflitos de naturezas diferentes.

Passo 2 — Suíte automatizada (30 s)

Terminal B:

```
python -m pytest -p no:warnings
```

```
AP2-demo

PS .venv\Scripts\python.exe -m pytest -p no:warnings
.....
40 passed in 0.56s
-
```

O que dizer: 40 testes cobrindo contrato e invariantes. Dois deles testam concorrência — e foram verificados desabilitando o lock do repositório, o que os faz falhar. Eles não passam por acaso.

Passo 3 — Servidor no ar (20 s)

Terminal A:

```
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

```
AP2-servidor2

PS .venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --no-use-colors

INFO: Started server process [32960]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
-
```

Deixe este terminal visível: a partir daqui ele mostra uma linha de log por requisição.

Se preferir `fastapi dev app/main.py` e ele falhar com `UnicodeEncodeError`, é o banner colorido da CLI contra o console em cp1252 — use o `uvicorn` acima.

Passo 4 — Criação de recurso (45 s)

Terminal B:

```
.\demo\01-criar-oficina.ps1
```



```
AP2-demo

PS .\demo\01-criar-oficina.ps1

Criacao de recurso -- 201 com Location e ETag

curl -i -X POST http://127.0.0.1:8000/oficinas -d @demo/corpos/oficina.json
HTTP/1.1 201 Created
date: Mon, 31 Aug 2026 23:40:49 GMT
server: uvicorn
content-length: 217
content-type: application/json
location: /oficinas/6
etag: "1"
x-request-id: dd267d5333b7

{"titulo":"Sistemas distribuidos na pratica","instrutor":"Helena Bastos","vagas_totais":2,"inicio":"2026-11-0180","status":"aberta","id":6,"versao":1,"vagas_ocupadas":0,"vagas_disponiveis":2}

Location diz onde o recurso passou a existir; ETag identifica esta versao.
```

O que dizer: três coisas na resposta. O **201** informa criação; o **location** diz onde o recurso passou a existir, para o cliente não precisar montar a URI a partir do corpo; e o **etag** identifica esta versão, que é o que torna possível detectar escrita concorrente depois.

Repare também em **vagas_ocupadas** e **vagas_disponiveis** : são calculados pelo servidor, não armazenados. Não existe contador para dessincronizar da lista.

Passo 5 — Conflito por unicidade (45 s)

Terminal B:

```
.\demo\02-inscricao-duplicada.ps1
```

```
AP2-demo

PS .\demo\02-inscricao-duplicada.ps1

Unicidade -- o mesmo e-mail nao se inscreve duas vezes

primeira inscricao:
curl -i -X POST http://127.0.0.1:8000/oficinas/1/inscricoes -d @demo/corpos/bruno.json
HTTP/1.1 201 Created

o mesmo e-mail, agora em maiusculas:
curl -i -X POST http://127.0.0.1:8000/oficinas/1/inscricoes -d @demo/corpos/bruno-maiusculo.json
HTTP/1.1 409 Conflict
date: Mon, 31 Aug 2026 23:40:55 GMT
server: uvicorn
content-length: 145
content-type: application/json
x-request-id: 504eacb819f2

{"erro":{"codigo":"inscricao_duplicada","mensagem":"bruno.lima@exemplo.com ja possui inscricao ativa nesta of
b819f2"}}

409: repetir este POST conflita em vez de duplicar.

-
```

O que dizer: a primeira inscrição é aceita; a segunda, com o mesmo e-mail em maiúsculas, recebe **409**. O e-mail é normalizado antes da comparação, senão bastaria trocar a caixa para furar a regra.

Este é o momento de mencionar o contraste: repetir `POST /oficinas` cria um segundo recurso, repetir este `POST` conflita. Nenhum dos dois é idempotente — a diferença é que aqui existe um invariante de unicidade que o servidor protege.

Passo 6 — Conflito por capacidade (30 s)

Terminal B:

```
.\demo\03-oficina-lotada.ps1
```

```
AP2-demo

PS .\demo\03-oficina-lotada.ps1

Capacidade -- a oficina 3 ja esta lotada

curl -i -X POST http://127.0.0.1:8000/oficinas/3/inscricoes -d @demo/corpos/carla.json
HTTP/1.1 409 Conflict
date: Mon, 31 Aug 2026 23:41:03 GMT
server: uvicorn
content-length: 115
content-type: application/json
x-request-id: 6caf6786e4f5

{"erro":{"codigo":"oficina_lotada","mensagem":"A oficina tem 2 vagas, todas ocupadas","request_id":"6caf6786e4f5"},
  "mensagem": "A oficina 3 ja esta lotada"}
409, e nao 422: a representacao esta correta, o estado e que nao permite.

-
```

O que dizer: 409 e não 422. A representação enviada está perfeita; o que impede é o estado do servidor. A mesma requisição teria sido aceita antes da última vaga acabar. É a distinção que mais se erra em API REST.

Passo 7 — Concorrência com ETag (60 s)

Terminal B:

```
.\demo\04-precondicao.ps1
```

```
AP2-demo

PS .\demo\04-precondicao.ps1

Concorrencia -- escrita exige a precondicao If-Match

sem If-Match:
curl -i -X PATCH http://127.0.0.1:8000/oficinas/1 -d @demo/corpos/titulo.json
HTTP/1.1 428 Precondition Required

com um ETag obsoleto:
curl -i -X PATCH http://127.0.0.1:8000/oficinas/1 -d @demo/corpos/titulo.json -H 'If-Match: "99"'
HTTP/1.1 412 Precondition Failed

com o ETag vigente:
curl -i -X PATCH http://127.0.0.1:8000/oficinas/1 -d @demo/corpos/titulo.json -H 'If-Match: "1"'
HTTP/1.1 200 OK
date: Mon, 31 Aug 2026 23:41:09 GMT
server: uvicorn
content-length: 215
content-type: application/json
etag: "2"
x-request-id: 2e0c28ddb576

428 exige a precondicao, 412 detecta escrita concorrente, 200 avanca o ETag.
```

O que dizer: os três casos em sequência.

- **428** — a escrita não trouxe `If-Match`. A precondição é exigida, não opcional: se fosse opcional, quem não a enviasse continuaria capaz de causar a perda que o mecanismo existe para impedir.
- **412** — trouxe um ETag que não corresponde à versão atual. Houve escrita concorrente.
- **200** — com o ETag vigente. Note o `etag` da resposta virar `"2"`: a versão avançou, e o `"1"` que outro cliente ainda tenha em mãos já não vale.

Passo 8 — Indisponibilidade com resposta (40 s)

Terminal B:

```
.\demo\05-manutencao.ps1
```

```
AP2-demo

PS .\demo\05-manutencao.ps1

Indisponibilidade -- 503 com Retry-After

ativando manutencao:
curl -i -X POST http://127.0.0.1:8000/admin/manutencao -d @demo/corpos/manutencao-on.json -H 'X-Token-Admin:
HTTP/1.1 200 OK

requisicao de dominio durante a manutencao:
curl -i -X GET http://127.0.0.1:8000/oficinas
HTTP/1.1 503 Service Unavailable
date: Mon, 31 Aug 2026 23:41:16 GMT
server: uvicorn
retry-after: 5
x-request-id: 7daef3f0bf33
content-length: 109
content-type: application/json

{"erro":{"codigo":"em_manutencao","mensagem":"Servico em manutencao programada","request_id":"7daef3f0bf33"}}

Ha resposta: status e Retry-After. Sem servidor, nao haveria resposta alguma.

-
```

O que dizer: 503 acompanhado de `retry-after: 5` . O cliente sabe que o serviço existe, está indisponível, e quando tentar de novo. Guarde este quadro: o passo 12 mostra o contraste.

Passo 9 — Timeout do cliente (40 s)

Terminal B:

```
python demo_timeout.py
```

```
AP2-demo

PS .venv\Scripts\python.exe demo_timeout.py

Rota que demora 2s, chamada com tres limites de timeout:

timeout=0.5s    -> Timeout: desistiu antes de qualquer resposta
timeout=1.0s    -> Timeout: desistiu antes de qualquer resposta
timeout=3.0s    -> HTTP 200 {'dormiu_s': 2.0}

Timeout nao e o mesmo que erro do servidor: aqui nao ha status nem
corpo para interpretar. Para uma escrita, o cliente fica sem saber se
a operacao foi aplicada -- e por isso que idempotencia importa.

-
```

O que dizer: os dois primeiros expiram, o terceiro completa. O ponto não é que o timeout funciona — é o que o cliente **não** sabe quando ele estoura: se o servidor executou a operação ou não. Para leitura isso é irrelevante; para escrita é o motivo pelo qual idempotência importa.

Passo 10 — Tabela de evidências (45 s)

Terminal B:

```
python cliente_testes.py
```

```
AP2-demo

PS .venv\Scripts\python.exe cliente_testes.py 2>&1 | Select-Object -Last 22

[ ok] Ocupar a unica vaga                esperado=201    obtido=201
[ ok] Inscrever com a oficina lotada      esperado=409    obtido=409
[ ok] Criar oficina em rascunho          esperado=201    obtido=201
[ ok] Inscrever em oficina nao aberta     esperado=409    obtido=409
[ ok] Marcar presença                    esperado=200    obtido=200
[ ok] Cancelar inscricao                 esperado=204    obtido=204
[ ok] Cancelar a mesma inscricao de novo esperado=204    obtido=204
[ ok] Reativar inscricao cancelada        esperado=409    obtido=409
[ ok] Cancelar a segunda inscricao       esperado=204    obtido=204
[ ok] Reler a oficina liberada            esperado=200    obtido=200
[ ok] Remover oficina sem inscritos       esperado=204    obtido=204
[ ok] Remover a mesma oficina de novo    esperado=404    obtido=404
[ ok] Rota de 2s com timeout de 0.5s      esperado=Timeout obtido=Timeout
[ ok] Rota de 2s com timeout de 1.0s      esperado=Timeout obtido=Timeout
[ ok] Rota de 2s com timeout de 3.0s      esperado=200    obtido=200
[ ok] Ativar manutencao sem token         esperado=401    obtido=401
[ ok] Ativar manutencao com token         esperado=200    obtido=200
[ ok] Listar oficinas em manutencao       esperado=503    obtido=503
[ ok] Desativar manutencao               esperado=200    obtido=200

Tabela gravada em D:\_Projetos\Faculdade\SistemasDistribuidos\Atividade_Prática_II\evidencias\tabela.md
46/46 cenarios conforme o esperado

-
```

O que dizer: 46 cenários executados contra o servidor real, e a tabela entregável é **gerada** por esta execução. Uma tabela escrita à mão descreve o que se acredita que a API faz; esta descreve o que ela fez.

Passo 11 — Concorrência real (60 s)

Terminal B:

```
python cliente_concorrente.py
```

```
AP2-demo
PS .venv\Scripts\python.exe cliente_concorrente.py 2>&1 | Select-Object -First 28

=====
Experimento 1 -- 12 clientes disputam 1 vaga
=====
201 Created : 1
409 Conflict : 11
vagas_ocupadas no servidor: 1
vagas_totais : 1

Resultado: invariante preservado
Exatamente um cliente conseguiu a vaga; os demais receberam um
conflito explícito em vez de uma inscrição que não caberia.

=====
Experimento 2 -- dois clientes editam a mesma oficina
=====
Cliente A leu a versão "1"
Cliente B leu a versão "1"

PATCH do cliente A -> 200
PATCH do cliente B -> 412 (precondicao_falhou)

instrutor no servidor: 'Alterado pelo cliente A'

Resultado: perda de atualização detectada
A alteração de A sobreviveu. B foi informado de que sua leitura
ficou obsoleta e pode reler antes de decidir -- em vez de apagar
silenciosamente o trabalho de A.
```

O que dizer — este é o passo mais forte da apresentação, com dois experimentos:

Doze clientes, uma vaga. Exatamente um 201 e onze 409, e o servidor termina com `vagas_ocupadas = 1`. Sem exclusão mútua, vários passariam pela verificação de capacidade antes que qualquer um gravasse.

Dois clientes editando. Ambos leem a versão `"1"`. O primeiro grava com 200, o segundo recebe 412. A alteração do primeiro sobreviveu, e o segundo foi informado em vez de apagar o trabalho dele silenciosamente.

Passo 12 — Falha de conectividade (40 s)

Encerre o servidor no **terminal A** com `Ctrl+C`, e no **terminal B**:

```
python cliente_testes.py --offline
```



```
AP2-demo

PS .venv\Scripts\python.exe cliente_testes.py --offline 2>&1 | Select-Object -Last 10

[ ok] Listar oficinas com o servidor desligado      esperado=ConnectionError obtido=ConnectionError
[ ok] Criar oficina com o servidor desligado        esperado=ConnectionError obtido=ConnectionError

Tabela gravada em D:\_Projetos\Faculdade\SistemasDistribuidos\Atividade_Prática_II\evidencias\tabela.md
2/2 cenários conforme o esperado

-
```

O que dizer: compare com o passo 8. Lá havia **503**: status, corpo e `Retry-After`. Aqui há `ConnectionError`: nenhuma resposta, nada para interpretar. Tratar os dois como "o serviço caiu" apaga a diferença — no primeiro caso existe um servidor vivo que escolheu recusar e disse quando voltar.

Passo 13 — Observabilidade (30 s)

Terminal B:

```
Get-Content evidencias\servidor.log -Tail 12 | ConvertFrom-Json | Select-Object request_id,
metodo, caminho, status, duracao_ms | Format-Table
```

```
AP2-demo

PS Get-Content evidencias\servidor.log -Tail 12 | ConvertFrom-Json | Select-Object request_id, metodo, caminho
at-Table -AutoSize

request_id  metodo  caminho  status  duracao_ms
-----
dd267d533b7 POST    /oficinas  201      3,12
de0b5186a207 POST    /oficinas/1/inscricoes  201      2,68
504eacb819f2 POST    /oficinas/1/inscricoes  409      0,62
6caf6786e4f5 POST    /oficinas/3/inscricoes  409      0,6
23a244acc1d2 PATCH    /oficinas/1  428      0,79
06266d41fff5 PATCH    /oficinas/1  412      0,59
2e0c28ddb576 PATCH    /oficinas/1  200      2,25

637f8e4d6b6b POST    /admin/manutencao  200      1,14
7daef3f0bf33 GET      /oficinas  503      0,02

93b0db763d76 POST    /admin/manutencao  200      0,92

-
```

O que dizer: a demonstração inteira aparece aqui — 201, 409, 428, 412, 200, 503 — cada requisição com identificador de correlação e duração. O `request_id` é o mesmo que volta no corpo do erro, ligando o que o cliente viu à linha do servidor. Nenhum token é registrado.

Se algo der errado

Sintoma	Causa provável	Saída
<code>ConnectionError</code> fora do passo 12	Servidor não está no ar	Suba o terminal A
<code>409</code> inesperado no passo 4 ou 5	Estado sujo de um ensaio anterior	<code>python seed.py</code> e reinicie o servidor
<code>412</code> onde se esperava <code>200</code>	O ETag mudou desde o ensaio	<code>curl -i http://127.0.0.1:8000/oficinas/1</code> e use o ETag atual
Porta 8000 ocupada	Servidor anterior não encerrado	Encerre o processo, ou use <code>--port 8001</code>
<code>UnicodeEncodeError</code> ao subir	Banner da CLI do FastAPI em cp1252	Use <code>python -m uvicorn ...</code>

Se o ao vivo falhar, as capturas deste documento são a evidência da execução — e `evidencias/tabela.md` traz os 48 cenários com status esperado e obtido.

Cobertura do checklist da especificação

Item do checklist	Passo
A API inicia a partir das instruções do README	3
Todos os endpoints obrigatórios foram testados	2 e 10
Evidências de pelo menos três cenários de erro	5, 6, 7, 8 e 12
Timeouts são definidos no cliente	9
Os resultados foram analisados, não apenas capturados	11 e 12, com <code>docs/analise.md</code>

Como estas capturas foram feitas

Os cinco scripts em `demo/` executam os passos de HTTP e imprimem o comando em forma legível antes de rodá-lo. Eles são reprodutíveis: qualquer pessoa com o repositório e o servidor no ar obtém as mesmas respostas.

6. Análise

Documento escrito depois da coleta de evidências, e não antes. As respostas citam resultados observados em `evidencias/tabela.md`, `evidencias/concorrencia.txt` e `evidencias/servidor.log`.

1. Quais operações são idempotentes e por quê?

Idempotência é uma propriedade do **efeito no estado do servidor**, não da resposta. Uma operação é idempotente quando executá-la N vezes deixa o recurso no mesmo estado em que uma única execução o deixaria. O status devolvido pode variar entre as tentativas sem que isso quebre a propriedade.

Operação	Idempotente	Observação
GET	Sim	Não altera estado; é também segura
PUT /oficinas/{id}	Sim, quanto ao estado final	A repetição exige o ETag vigente
PATCH /oficinas/{id}	Depende do conteúdo	Aqui sim: todos os campos são atribuições absolutas
DELETE /oficinas/{id}	Sim, quanto ao efeito	Segunda chamada responde 404
DELETE /inscricoes/{id}	Sim, integralmente	Segunda chamada responde 204
POST /oficinas	Não	Cada chamada cria um recurso distinto
POST /oficinas/{id}/inscricoes	Não	A repetição conflita em vez de duplicar

O que as evidências mostram

DELETE de oficina (cenários 38 e 39 da tabela): a primeira chamada devolve 204, a segunda devolve 404. O efeito é idempotente — o recurso está ausente nos dois casos —, mas os status diferem porque **cada resposta descreve a requisição que a originou**, e não o estado final. A segunda requisição de fato não encontrou nada para remover, e dizer isso é mais informativo do que mentir 204.

DELETE de inscrição (cenários 33 e 34): as duas chamadas devolvem 204. A diferença vem da modelagem: cancelar não apaga o registro, apenas muda seu status para `cancelada`. Como o registro permanece, a segunda chamada encontra o recurso e reaplica uma operação que já estava aplicada. É o caso mais limpo de idempotência da API — mesmo efeito e mesmo status.

Os dois comportamentos de POST. Esta é a observação mais interessante que a API produz, porque os dois casos convivem na mesma aplicação:

- Repetir `POST /oficinas` (cenário 3) devolve **201** e cria um segundo recurso, com identificador diferente. É a não-idempotência clássica.

- Repetir `POST /oficinas/{id}/inscricoes` (cenário 22) devolve **409**.

Nenhum dos dois é idempotente. A diferença não está no método, mas no **invariante do recurso**: inscrições têm uma regra de unicidade — um e-mail ativo por oficina — e oficinas não têm. Onde existe invariante de unicidade, a repetição encontra o conflito; onde não existe, ela duplica. Isso mostra que a não-idempotência do POST não é um comportamento único, e sim a ausência de garantia: o que acontece na repetição depende inteiramente do domínio.

Escritas repetidas com o mesmo ETag. Reenviar uma alteração com o ETag já consumido devolve **412**, e não 200 — observado no cenário 18, um `PATCH`, e válido igualmente para `PUT`. Isso não viola a idempotência: o teste `test_put_identicico_repetido_leva_ao_mesmo_estado` demonstra que, relendo o ETag entre as tentativas, a representação resultante é idêntica em todos os campos exceto `versao`. A condição é uma camada de controle de concorrência sobreposta ao método; ela restringe *quando* a operação é aceita, sem alterar *qual* estado ela produz.

2. Qual a diferença entre "servidor respondeu erro" e "cliente não conseguiu obter resposta"?

A diferença é a existência de informação.

Quando o servidor responde um erro, há uma mensagem HTTP completa: status, cabeçalhos e corpo. O cliente sabe que a requisição chegou, foi interpretada e recebeu um veredito. Pode agir sobre ele.

Quando o cliente não obtém resposta, ele não sabe praticamente nada. Não sabe se a requisição chegou ao servidor, se chegou e foi processada, se foi processada e a resposta se perdeu no caminho, ou se nunca saiu da máquina dele.

Três situações que as evidências separam

Situação	O que o cliente recebe	O que ele sabe
503 (cenário 45)	Status, corpo e <code>Retry-After: 5</code>	O serviço existe, está indisponível e sugere quando tentar de novo
Timeout (cenários 40–41)	Exceção <code>Timeout</code>	Desistiu de esperar. Não sabe se a operação ocorreu
ConnectionError (cenários 47–48)	Exceção <code>ConnectionError</code>	Não houve conexão. A operação quase certamente não ocorreu

O par 503 versus `ConnectionError` é frequentemente tratado como equivalente — "o serviço está fora" — mas são coisas diferentes na prática. Com 503 há um servidor vivo que escolheu recusar, e o `Retry-After` é uma instrução dele. Com `ConnectionError` não há interlocutor: qualquer política de reenvio é uma suposição do cliente.

O **timeout é o caso mais difícil dos três**, e é o que justifica a atenção da disciplina à idempotência. O cliente desistiu, mas o servidor pode ter executado a operação inteira e a resposta pode ter chegado tarde demais. Para uma leitura, isso é irrelevante: basta repetir. Para uma escrita, é decisivo. Repetir um `PUT`

após timeout é seguro, porque o estado final é o mesmo tenha a primeira tentativa ocorrido ou não. Repetir um `POST /oficinas` pode criar duas oficinas, sendo que o usuário pediu uma. Repetir um `POST` de inscrição é seguro por acidente: se a primeira tentativa passou, a segunda recebe 409 e o estado permanece correto — a regra de unicidade acaba funcionando como proteção contra reenvio.

É por isso que todo `requests` em `cliente_testes.py` tem `timeout` explícito. Sem ele a biblioteca espera indefinidamente, e um cliente travado é pior que um cliente com erro: a falha deixa de ser observável e se propaga como lentidão para quem depende dele.

3. Que decisões da API aumentam ou reduzem o acoplamento entre cliente e servidor?

Decisões que reduzem acoplamento

Operações de item da inscrição em primeiro nível. Criar e listar são aninhadas (`/oficinas/{id}/inscricoes`), mas obter, alterar e cancelar usam `/inscricoes/{id}`. Se o cancelamento exigisse o caminho completo, todo cliente que possuísse apenas o identificador da inscrição precisaria também guardar o da oficina. A URI passaria a codificar uma informação que o servidor já conhece.

Location nas criações. O servidor informa onde o recurso passou a existir. Sem esse cabeçalho o cliente teria de montar a URI a partir do corpo, o que o acopla ao formato do identificador — e uma futura troca de inteiro sequencial por UUID quebraria todos os clientes.

Listagens em envelope, não em array cru. `GET /oficinas` devolve `{total, limite, offset, itens}`. Acrescentar metadados a um objeto é uma mudança compatível; trocar um array por um objeto não é. O envelope reserva o espaço de evolução desde o início.

Envelope de erro estável. Todo erro tem a forma `{erro: {codigo, mensagem, request_id}}`. O cliente decide o que fazer pelo `codigo`, um identificador estável, sem inspecionar o texto da `mensagem` — que pode ser reescrito a qualquer momento sem quebrar ninguém.

Códigos de status distintos por causa. Separar 404, 409, 412, 422 e 428 permite ao cliente reagir apropriadamente a cada um. Uma API que devolvesse 400 para tudo obrigaria o cliente a fazer análise textual da mensagem para descobrir o que houve.

vagas_disponiveis calculado pelo servidor. O cliente não precisa conhecer a regra de quais status ocupam vaga. Se essa regra mudar, nenhum cliente muda junto.

Decisões que aumentam acoplamento

If-Match obrigatório nas escritas de oficina. O cliente é obrigado a ler antes de escrever e a manter o ETag entre as duas operações. Isso é acoplamento real, adotado deliberadamente: o preço de não ter é a perda silenciosa de atualização, demonstrada na questão 4. A alternativa — condição opcional — foi descartada porque tornaria a proteção efetiva apenas para os clientes que já sabem que ela existe, isto é, justamente os que menos precisam dela.

extra="forbid" nos modelos de entrada. Um campo não previsto vira 422 em vez de ser ignorado. Acopla o cliente ao conjunto exato de campos aceitos, mas evita a falha silenciosa em que ele acredita ter

feito uma alteração que nunca aconteceu. Erro explícito é melhor que silêncio, e a assimetria de custo entre os dois justifica a rigidez.

Ciclo de vida de status validado no servidor. As transições permitidas são uma regra que o cliente precisa conhecer para construir uma interface coerente. O acoplamento é aceito porque a alternativa — deixar qualquer transição passar — permitiria estados sem sentido, como uma oficina cancelada que volta a aceitar inscrições.

Ausência de hipermídia. A API não devolve links de navegação. Os clientes constroem URIs a partir de conhecimento prévio da estrutura, o que os acopla ao desenho dos caminhos. É a decisão mais distante do REST original, tomada por proporcionalidade: hipermídia se paga em APIs públicas com muitos consumidores independentes, não num laboratório com um cliente conhecido.

4. Como evitar perda de atualização quando dois clientes editam o mesmo recurso?

O problema

Dois clientes leem a mesma representação, ambos a modificam e ambos gravam. A segunda gravação sobrescreve a primeira. Ninguém recebe erro, e a alteração do primeiro desaparece sem deixar rastro. É a pior classe de falha: silenciosa e indistinguível de funcionamento normal.

A solução adotada: controle otimista com ETag e If-Match

Cada oficina tem um contador `versao`, incrementado a cada escrita e exposto como `ETag`. Escritas exigem `If-Match`. O servidor compara a versão informada com a vigente e recusa quando divergem.

`evidencias/concorrencia.txt` registra o experimento:

```
Cliente A leu a versao "1"
Cliente B leu a versao "1"

PATCH do cliente A -> 200
PATCH do cliente B -> 412 (precondicao_falhou)

instrutor no servidor: 'Alterado pelo cliente A'
```

A alteração de A sobreviveu. B foi informado de que sua leitura ficou obsoleta e pôde reler antes de decidir — na repetição com o ETag vigente, sua alteração foi aceita com 200. A perda de atualização deixou de ser silenciosa e virou um erro que o cliente pode tratar.

Ausência da precondição recebe **428 Precondition Required**, e não é tratada como permissão para sobrescrever. Essa escolha é o que dá valor ao mecanismo: se o `If-Match` fosse opcional, um cliente que simplesmente não o enviasse continuaria capaz de causar a perda que o mecanismo existe para impedir.

Por que otimista e não pessimista

O controle pessimista — travar o recurso na leitura e liberar na escrita — resolveria o mesmo problema, mas transfere um novo: um cliente que lê e nunca grava mantém o recurso travado. Seria preciso

expiração de trava, renovação e tratamento de trava órfã. O controle otimista não bloqueia nada e só custa uma releitura no caso raro de conflito real. Em recursos com muito mais leituras que escritas concorrentes, essa é a troca certa.

O segundo mecanismo: exclusão mútua no repositório

O ETag protege contra escritas baseadas em leituras obsoletas. Ele **não** protege o outro problema: `POST` de inscrição não tem representação prévia para validar, e portanto não tem ETag. Ali a proteção precisa vir da atomicidade.

`Repositorio` executa todo ciclo ler-verificar-escrever sob um `RLock`. Verificar se há vaga e inserir a inscrição são operações distintas, e sem exclusão mútua dois clientes podem passar pela verificação antes que qualquer um grave — o padrão *check-then-act*. O experimento com 12 clientes disputando uma vaga confirma o resultado esperado:

```
201 Created : 1
409 Conflict : 11
vagas_ocupadas no servidor: 1
```

Que o teste tem valor foi verificado diretamente: substituindo o `RLock` por um contexto vazio e inserindo uma pausa entre a verificação e a inserção, tanto `test_capacidade_nunca_e_excedida_sob_concorrencia` quanto `test_identificadores_nao_se_repetem_sob_concorrencia` passam a falhar. O teste não passa por acaso — ele passa por causa do lock.

A conferência do `If-Match` também acontece **dentro** do lock. Fora dele, o próprio controle de versão teria a corrida que existe para eliminar: dois clientes poderiam validar a mesma versão antes que qualquer um a incrementasse.

Limite conhecido

O `RLock` coordena apenas threads do mesmo processo. Duas instâncias do servidor sobre o mesmo arquivo não estariam protegidas: cada uma teria seu próprio lock e sua própria cópia do estado em memória. Resolver isso exigiria trava de arquivo do sistema operacional, um armazenamento com transações reais, ou mover o invariante para uma restrição do banco.

O ETag, por ser um mecanismo de protocolo e não de processo, continuaria funcionando parcialmente entre instâncias — mas só se a versão fosse lida e gravada atomicamente no armazenamento compartilhado, o que recai no mesmo requisito. Vale registrar a conclusão geral: **HTTP oferece o vocabulário para expressar o conflito, não a garantia de detectá-lo**. Quem detecta é a camada de armazenamento.

Justificativa das escolhas de URI, método e status

URIs

Substantivos no plural, hierarquia refletindo a dependência real entre as entidades, e nenhum verbo no caminho. Cancelar uma inscrição é `DELETE /inscricoes/{id}`, e não `POST /inscricoes/{id}/cancelar`: a

combinação de recurso e método já expressa a semântica.

A exceção consciente é `/admin/manutencao`, que não é um recurso do domínio e sim uma chave operacional. Está isolada sob um prefixo próprio justamente para não se confundir com o modelo de negócio.

Métodos

`PUT` e `PATCH` coexistem porque servem a necessidades diferentes. `PUT` carrega a representação inteira e é a operação natural de um formulário de edição completo. `PATCH` carrega apenas o que muda, e é o que um cliente usa para alterar só o status ou só o número de vagas, sem precisar conhecer nem reenviar os demais campos.

`DELETE` de inscrição faz cancelamento lógico em vez de remoção física. Do ponto de vista do cliente a semântica é a esperada — a vaga é liberada e a inscrição deixa de valer —, e o histórico de desistências é preservado.

Status

Código	Quando	Por quê
200	Operação concluída com corpo	—
201	Recurso criado	Acompanha <code>Location</code>
204	Sucesso sem corpo	Remoção e cancelamento
304	Representação inalterada	Resposta a <code>If-None-Match</code>
401	Token administrativo ausente ou inválido	Autenticação, não autorização
404	Recurso inexistente	Inclusive ao listar filhos de pai inexistente
409	Conflito com o estado atual	As seis regras de domínio
412	<code>If-Match</code> divergente	Houve escrita concorrente
422	Representação semanticamente inválida	Padrão do FastAPI
428	<code>If-Match</code> ausente	A precondição é exigida, não opcional
500	Falha não prevista	Sem expor detalhes internos
503	Manutenção	Acompanha <code>Retry-After</code>

A distinção mais importante é entre **422** e **409**, e é a que mais se erra na prática. 422 fala da representação enviada: ela está malformada segundo as regras do modelo e seria rejeitada em qualquer instante. 409 fala do estado do servidor: a requisição está correta em si e conflita com a situação atual. A mesma inscrição rejeitada com 409 por lotação seria aceita um segundo antes, sem que nada nela mudasse.

404 ao listar inscrições de uma oficina inexistente, em vez de lista vazia, é outra decisão deliberada. São situações distintas — "esta oficina não tem inscritos" e "esta oficina não existe" — e confundi-las esconderia do cliente que ele está consultando um identificador errado.

Considerações finais

Três limites desta implementação merecem registro explícito, porque uma análise que só descreve acertos é incompleta:

1. **A persistência não é transacional.** Escritas são atômicas por arquivo, não por operação lógica. Uma falha entre duas gravações relacionadas deixaria estado inconsistente. O lock reduz a janela, mas não a elimina.
2. **A coordenação não atravessa processos.** Está detalhado na questão 4. É o limite mais relevante do ponto de vista de sistemas distribuídos: a solução funciona porque há uma única instância.
3. **Não há autenticação de participantes.** Qualquer cliente pode cancelar qualquer inscrição. O token cobre apenas o endpoint administrativo. Numa aplicação real, a posse da inscrição precisaria ser verificada — e o status correto para a recusa seria 403, não 409.

Nenhum dos três é resolvido por usar REST ou HTTP. É a observação que atravessa o capítulo 4 da apostila e que esta atividade confirma na prática: o protocolo oferece vocabulário e semântica para expressar problemas de sistemas distribuídos, mas resolvê-los continua sendo trabalho da aplicação e da camada de armazenamento.